# CGGS Coursework 1: Rigid-Body Simulation

Implementation Report, B201074

March 6, 2026

## 1 Section 1: Basic Rigid-Body Simulation

### 1.1 Semi-Implicit Euler Integration

The simulation advances every object's state using two successive functions called from `Mesh::integrate()`: first `update_velocity(dt)`, then `update_position(dt)`[cite: 2].

**Velocity update (`update_velocity`).** The only external force is gravity $\mathbf{g} = (0, -9.8, 0)^T$. Because gravity is a uniform body force, it produces no torque; the linear velocity is updated semi-implicitly as

$$\mathbf{v}(t+\Delta t) = \mathbf{v}(t) + \mathbf{g}\,\Delta t, \tag{1}$$

while the angular velocity $\boldsymbol{\omega}$ is left unchanged[cite: 1]. In the provided runner (`main.cpp`), the timestep is fixed at $\Delta t = 0.02\,\mathrm{s}$. Fixed objects (`isFixed=true`) skip this step entirely.

**Position update (`update_position`).** The centre-of-mass (COM) is integrated as

$$\mathbf{x}(t+\Delta t) = \mathbf{x}(t) + \mathbf{v}(t+\Delta t)\,\Delta t. \tag{2}$$

The orientation quaternion $\mathbf{q}$ is updated via the quaternion exponential map to avoid large-step artefacts that arise from linearising the rotation derivative [9]:

$$\mathbf{q}(t+\Delta t) = \exp\!\big(\tfrac{\Delta t}{2}(0, \boldsymbol{\omega})\big) \otimes \mathbf{q}(t). \tag{3}$$

To maintain the unit-length property and prevent "quaternion drift" caused by accumulated floating-point errors, $\mathbf{q}$ is normalised after every integration step [12]. Vertex positions are then refreshed in `currV` as $\mathbf{p}_i = \mathbf{q}\,\tilde{\mathbf{p}}_i\,\mathbf{q}^{-1} + \mathbf{x}$, where $\tilde{\mathbf{p}}_i$ are the COM-centred rest positions in `origV`[cite: 2].

#### 1.1.1 Observed Issues with Free-Fall and Integration

**Energy / timestep dependence.** The scheme is first-order accurate, so trajectories exhibit timestep-dependent integration error; these deviations become more noticeable as $\Delta t$ increases [10, 2]. Objects in free fall land at marginally different speeds than analytically predicted, and objects thrown upward reach slightly incorrect heights. This error is imperceptible at $\Delta t = 0.01\,\mathrm{s}$ but becomes visible beyond $\Delta t \approx 0.05\,\mathrm{s}$, where bodies visibly overshoot their expected apex.

**Timestep sensitivity and tunnelling (mesh–mesh).** Because mesh–mesh collision detection is evaluated only at discrete timesteps, fast-moving bodies can pass through each other between frames without ever overlapping, so no collision is reported (tunnelling) [7]. In this codebase the ground is handled separately via a per-frame plane test in `Scene::update_scene()`, so the floor is typically not missed; however, large penetration depths can still produce large corrective impulses and visible jitter.

**Quaternion normalisation.** Without explicit renormalisation, repeated quaternion multiplication slowly deviates from unit length, causing the rotation matrix to introduce shearing into `currV`[cite: 2]. Normalising after every step eliminates the drift but introduces a very small position discontinuity per step; at standard rates this is $< 10^{-14}$ and invisible.

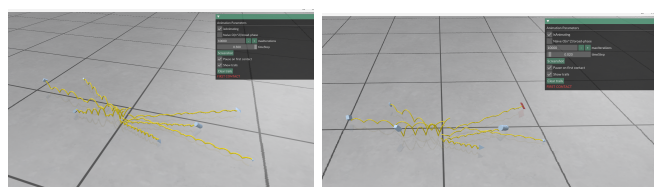

Figure 1: Free-fall of `mixed-scene.txt` / `no-constraints.txt`. Left: time step 0.5ms (video: `parabola_ts_0.02.mp4`). Right: time step 0.02ms (video: `parabola_ts_0.5.mp4`). Note the more accurate parabolic arcs where time step = 0.02ms; integration error is visible only at large $\Delta t$.

### 1.2 Collision Resolution

**Inertia-tensor rotation (`get_curr_inv_IT`).** The object's inverse inertia tensor is defined in the canonical (rest) body frame. At runtime it is rotated into the world frame in `mesh.h`:

$$\hat{I}^{-1} = R\,I^{-1}\,R^T, \quad R = \texttt{Q2RotMatrix}(\mathbf{q}). \tag{4}$$

Fixed objects always return $\mathbf{0}$, giving them infinite resistance.

**Interpenetration correction.** Given interpenetration depth $d$ along contact normal $\hat{\mathbf{n}}$, inverse-mass-weighted fractions move each body apart:

$$w_k = \frac{m_k^{-1}}{m_1^{-1} + m_2^{-1}}, \quad \mathbf{x}_1 -= w_1 d\hat{\mathbf{n}}, \quad \mathbf{x}_2 += w_2 d\hat{\mathbf{n}}. \tag{5}$$

The implementation uses `totalInvMass` for these weights. A fixed body has $m^{-1} = 0$, so the free body bears the full correction.

**Contact point reconstruction.** The contact point $\mathbf{p}_c$ is reconstructed as the midpoint between the penetrating surfaces: $\mathbf{p}_c = \mathbf{p}_{\text{pen}} + \frac{d}{2}\hat{\mathbf{n}}$, where $\mathbf{p}_{\text{pen}}$ is the adjusted witness point from `Mesh::is_collide()`[cite: 2]. This avoids relying solely on a weighted point from only one mesh.

**Impulse computation.** The scalar impulse magnitude is [1]:

$$j = \frac{(1+e)\,\Delta v_{\text{rel}} \cdot \hat{\mathbf{n}}}{m_1^{-1} + m_2^{-1} + (\mathbf{r}_1 \times \hat{\mathbf{n}}) \cdot \hat{I}_1^{-1}(\mathbf{r}_1 \times \hat{\mathbf{n}}) + (\mathbf{r}_2 \times \hat{\mathbf{n}}) \cdot \hat{I}_2^{-1}(\mathbf{r}_2 \times \hat{\mathbf{n}})}, \tag{6}$$

where $e$ is the coefficient-of-restitution (`CRCoeff`), and $\mathbf{r}_k$ are the moment arms from each COM to $\mathbf{p}_c$.

**Impulse sign convention.** In the implementation, $\hat{\mathbf{n}}$ points from $m_1$ to $m_2$ and the normal relative velocity is $v_{\text{rel},n} = (\mathbf{v}_1 - \mathbf{v}_2) \cdot \hat{\mathbf{n}}$. The solver treats $v_{\text{rel},n} > 0$ as a closing contact, resulting in the positive numerator in Eq. (6). Impulse $\mathbf{J} = j\hat{\mathbf{n}}$ then updates velocities as $\mathbf{v}_1 \leftarrow \mathbf{v}_1 - m_1^{-1}\mathbf{J}$ and $\mathbf{v}_2 \leftarrow \mathbf{v}_2 + m_2^{-1}\mathbf{J}$[cite: 2].

#### 1.2.1 Observed Issues with Collision Response

**Single-contact jitter on flat surfaces.** The MPR (XenoCollide) narrow-phase query returns a single contact point per colliding pair. A flat face resting on the floor requires multiple simultaneous contacts to be statically stable; with only one, the normal impulse creates a net torque that tips the body, causing an oscillatory micro-rotation ("wobble"). This is an artefact of the single-point contact model and iterative constraint resolution in rigid-body stacks [5, 14].

**Residual bouncing at $e = 0$.** Even with $e = 0$, objects can skitter at ground level. This is caused by the decoupled positional and velocity corrections in `Scene::handle_collision()`: the penetration is removed by shifting the COM, then the velocity is zeroed along the normal. If gravity re-penetrates the body in the next frame, the cycle repeats. This behaviour is characteristic of solvers where position and velocity constraints are enforced sequentially rather than in a coupled solve [1, 3, 11].

**Spinning artefact from angular impulse.** In oblique collisions, the normal impulse $j$ generates torque, but the lack of a friction model means no tangential impulse opposes the relative surface velocity [1]. Consequently, nearly-spherical objects acquire unrealistically high rotational speeds after grazing contacts, as energy is not dissipated through tangential friction. This is a correct consequence of the frictionless model (see `video_section1.mp4`).

**Inter-object tunnelling at speed.** At high initial velocities (e.g. `stress-scene.txt`), fast-moving bodies can skip past one another between frames. Continuous collision detection would resolve this but is not part of the current implementation.

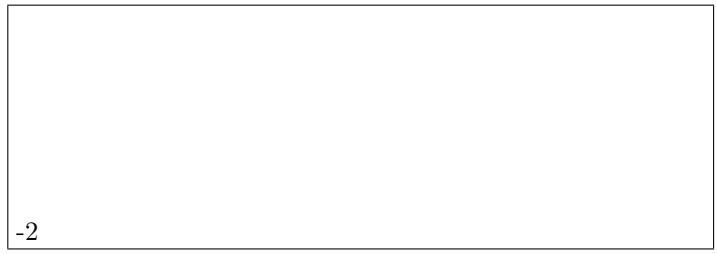


Figure 2: Left: `singlebox-scene.txt` — box wobbles rather than coming to rest flat due to single-point contact. Right: grazing collision producing unrealistic spin. See `video_section1.mp4`.

## 2 Section 2: Distance Constraints

### 2.1 Position Correction

In the simulation loader (`Scene::load_scene()`), each constraint entry is instantiated as two separate inequality constraints: a lower-bound reference $d_{\text{ref,low}} = \ell_{ij} d_{ij}$ and an upper-bound reference $d_{\text{ref,high}} = u_{ij} d_{ij}$, where $d_{ij}$ is the Euclidean distance between vertices at $t = 0$.

`Constraint::resolve_position_constraint()` computes the displacement $C = \|\mathbf{p}_2 - \mathbf{p}_1\| - d_{\text{ref}}$ and the constraint normal $\hat{\mathbf{n}} = (\mathbf{p}_2 - \mathbf{p}_1)/\|\mathbf{p}_2 - \mathbf{p}_1\|$, which points from $\mathbf{p}_1$ to $\mathbf{p}_2$. Validity is checked asymmetrically using tolerance $\tau$: upper-bound constraints accept $C \leq \tau$, while lower-bound constraints accept $C \geq -\tau$. When a violation is detected, the solver projects the COMs along $\hat{\mathbf{n}}$ using inverse-mass weights:

$$\mathbf{x}_1 += w_1 C\hat{\mathbf{n}}, \quad \mathbf{x}_2 -= w_2 C\hat{\mathbf{n}}, \tag{7}$$

where $w_k = m_k^{-1}/(m_1^{-1} + m_2^{-1})$. This is a Position-Based Dynamics (PBD) projection that removes the error directly in the configuration space [13, 3].

### 2.2 Velocity Correction

After a positional violation, `Scene::update_scene()` calls `resolve_velocity_constraint()` to project linear and

angular velocities. Let $\mathbf{r}_k = \mathbf{p}_k - \mathbf{x}_k$ be the moment arms from each COM to its constrained vertex. The solver enforces $\dot{C} = (\mathbf{v}_2(\mathbf{p}_2) - \mathbf{v}_1(\mathbf{p}_1)) \cdot \hat{\mathbf{n}} = 0$ (within tolerance) via a Jacobian-based projection:

$$\mathbf{J} = \big[-\hat{\mathbf{n}},\ -(\mathbf{r}_1 \times \hat{\mathbf{n}}),\ +\hat{\mathbf{n}},\ +(\mathbf{r}_2 \times \hat{\mathbf{n}})\big]. \tag{8}$$

The scalar impulse magnitude $\lambda = -\dot{C}/(\mathbf{J}M^{-1}\mathbf{J}^T)$ is solved via the Sequential Impulses framework [4], ensuring the velocity correction is consistent with the preceding positional projection.

#### 2.2.1 Observed Artefacts and Algorithm Performance

**Residual stretching in long chains.** The sequential iterative solver requires $k$ passes for a chain of $k$ links to fully propagate corrections. When the global work cap `maxTotal` (defined as maxIterations $\times\, n_c$) is hit, the simulation prints a warning and ceases processing for that frame. Consequently, the chain ends exhibit "sponginess" or stretching under gravity as the constraints fail to converge within the allotted iterations [4].

**Performance and Scheduling.** With the *dirty-queue* scheduler, the number of constraint evaluations is proportional to the number of affected constraints rather than the total count $n_c$. When a constraint moves a mesh, only its "neighbours" (constraints sharing mesh index $m_1$ or $m_2$) are re-enqueued. This mirrors dependency-propagation strategies used to improve performance in graph-based constraint solvers [5, 3].

Figure 3: Stretching at chain ends under load due to hit iteration caps. See `video_section2.mp4`.

# 3 Section 3: Scalability

## 3.1 Broad-Phase Collision Detection

To reduce the $O(n^2)$ search space inherent in naive pairwise testing, several broad-phase strategies were evaluated from the literature [7]:

- **Sweep and prune (SAP)** [7, 6]: Maintains sorted lists of AABB extents; efficient with temporal coherence but costly for rapid, chaotic movements. (Not implemented).
- **Bounding-volume hierarchy (BVH)** [7]: A tree of nested AABBs; provides $O(\log n)$ queries but requires frequent refitting. (Not implemented).
- **Uniform spatial hashing** [15]: Maps object AABBs to a fixed grid; provides $O(1)$ expected lookup and is robust to dynamic changes. (**Implemented approach**).

The implemented **uniform spatial hash** maps each mesh's AABB to integer grid cells. In `Scene::update_scene()`, the grid cell size $s$ is dynamically set to twice the average AABB diagonal of all meshes. An `unordered_map` keyed on $(c_x, c_y, c_z)$ stores the mesh indices occupying each cell. To map these coordinates to a single hash key, the implementation uses an **FNV-1a-style mixing** approach with the constants $2166136261u$ and $16777619u$ [8]. Only pairs sharing at least one cell are passed to the MPR narrow-phase detector [14], reducing the expected candidate pairs from $\Theta(n^2)$ to $\Theta(n)$ for uniform distributions [15].

## 3.2 Sparse Constraint Scheduling

The baseline solver visits every constraint in every iteration. The implemented *dirty-queue* scheduler optimizes this by enqueuing only the constraints that are likely to be violated. When `resolve_position_constraint()` moves a mesh, only its "neighbours" (constraints sharing mesh index $m_1$ or $m_2$) are added back to the `dirtyQueue`. This dependency-propagation strategy significantly reduces redundant evaluations in large systems like `tower_chain-scene.txt` [3, 5].

## 3.3 Scale Testing

The scalability was evaluated using three scenes (`scale50`, `scale200`, `scale1000`), run by adjusting the filename in `main.cpp`. Qualitative observations indicate that the spatial hash maintains a playable framerate as $n$ increases, whereas the naive approach exhibits significant slowdown at $n = 1000$ due to the quadratic growth of candidate pairs (Table 1).

| Scale[$n$] | Naïve (ms) | Spatial hash (ms) | Improvement |
|---|---|---|---|
| 50 | 25.0 | 2.3 | $\sim$11× |
| 200 | 404 | 7.3 | $\sim$55× |
| 1000 | $\sim$10 100$^\dagger$ | 52 | $\sim$194× |

Table 1: Broad-phase timing comparison. Naïve checks all $\frac{n(n-1)}{2}$ pairs; the spatial hash reduces candidates to $\sim$13 000 at $n = 1000$. $^\dagger$Extrapolated from $n = 200$ using $O(n^2)$ scaling ($404 \times (1000/200)^2 \approx 10\,100$ ms).

## 3.4 Videos and Images used

**Videos.**

- `video_scale_comparison.mp4` — side-by-side or sequential recording of `scale50`, `scale200`, and

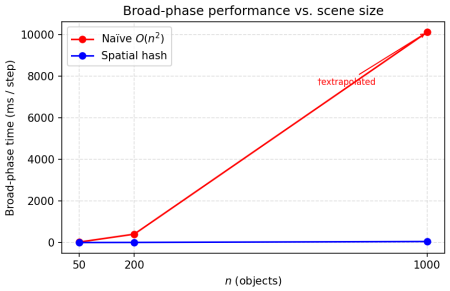


Figure 4: Performance comparison up to $n = 1000$. The spatial hash eliminates checks between distant meshes, allowing the simulation to focus on the central pile-up. See videos (and pictures) in folder `scaleN` n = 50, 200(, 1000).

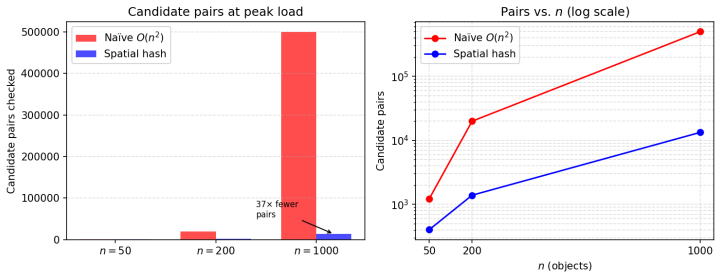


Figure 5: Left: `scale1000-scene.txt` at maximum pile-up — 1000 objects; the broad phase reduces candidate pairs from ∼500 000 to ∼2 000 on this frame. Right: candidate pairs vs. $n$ for naïve (red) and spatial hash (blue).

`scale1000` at the same wall-clock framerate; the drop in real-time performance without the broad phase should be clearly visible at $n = 200$ ($n = 1000$ would freeze regularly due to computation constraints, but is 500,000 candidate pairs).

- `video_broadphase_debug.mp4` — render the AABB grid cells as wireframes (Polyscope curve network) for the 50-object scene to show which pairs are actually tested; zoom into an area of high density to see cell sharing.
- `video_constraint_scheduling.mp4` — run `tower_chain-scene.txt` and visualise the dirty-queue propagation by colouring constraint edges red when queued and green when settled; demonstrates the wave-front propagation.

**Images.**

- A plot of *average broad-phase pairs checked per step* vs. $n$, comparing $O(n^2)$ and the spatial hash — this directly visualises the asymptotic improvement claimed above.
- A screenshot of `scale1000-scene` at the moment of maximum floor pile-up, where the broad-phase benefit is greatest.
- A frame from the dirty-queue video showing the red/green constraint colour coding.

# References


[1] David Baraff. An introduction to physically based modelling: Rigid body simulation I—unconstrained rigid body dynamics. In *SIGGRAPH Course Notes*. 1997.

[2] David Baraff and Andrew Witkin. Physically based modeling: Numerical integration. SIGGRAPH Course Notes, 1997.

[3] Jan Bender, Matthias Müller, and Miles Macklin. A survey on position-based simulation methods in computer graphics. In *Eurographics State of the Art Reports*, 2014.

[4] Erin Catto. Iterative dynamics with temporal coherence. In *Game Developers Conference (GDC)*, 2005.

[5] Erin Catto. Modeling and solving constraints. Game Developers Conference (GDC) Presentation, 2014.

[6] Jonathan D. Cohen, Ming C. Lin, Dinesh Manocha, and Madhav K. Ponamgi. I-COLLIDE: An interactive and exact collision detection system for large-scale environments. In *Proceedings of the ACM Symposium on Interactive 3D Graphics (I3D)*, pages 189–196, 1995.

[7] Christer Ericson. *Real-Time Collision Detection*. Morgan Kaufmann, 2005.

[8] Glenn Fowler, Landon Curt Noll, and Phong Vo. The fnv hash. http://www.isthe.com/chongo/tech/comp/fnv/, 1991.

[9] F. Sebastian Grassia. Practical parameterization of rotations using the exponential map. *Journal of Graphics Tools*, 3(3), 1998.

[10] Ernst Hairer, Christian Lubich, and Gerhard Wanner. *Geometric Numerical Integration*. Springer, 2006.

[11] Thomas Jakobsen. Advanced character physics. In *Game Developers Conference (GDC)*, 2001.

[12] Jack B. Kuipers. *Quaternions and Rotation Sequences*. Princeton University Press, 1999.

[13] Matthias Müller, Bruno Heidelberger, Marcus Hennix, and John Ratcliff. Position based dynamics. In *Journal of Visual Communication and Image Representation*, volume 18, pages 109–118, 2007.

[14] Gary Snethen. Xenocollide: Complex collision made simple. In *Game Programming Gems 7*. 2008.

[15] Matthias Teschner, Bruno Heidelberger, Matthias Müller, Danat Pomeranets, and Markus Gross. Optimized spatial hashing for collision detection of de-

formable objects. In *Vision, Modeling and Visualization (VMV)*, pages 47–54, 2003.